Motivation

I am very interested in politics and how different factors affect the standing of individual politicians as well as parties as a whole. Probably the most important and heavily tracked is the President. Our current President has a multitude of issues that affect the way voters view him. I want to see how much the Legislative Branch affects the approval rating of our Executive using Congressional voting information compared to the approval rating of President Trump.

Initial Set-up

The next two cells contain some initial operations and functions I used frequently to analyze the data.

1. The first cell imports all the libraries needed to accomplish my analysis. It also extracts today's date as a number for determining the default end date of my functions.
2. The second cell contains my functions that do the following:

    - Convert a numeric date to a standard date
    - Graph the approval rating of the President next to the aggregate Trump Score
    - Aggregate the Trump Score with different parameters for different analyses
    - Extract the mean, median, and mode of a list
    - Calculate the average approval rating over a specified number of days before and after a base date

In [1]:
```
import pandas as pd
import datetime
import matplotlib.pyplot as plt
dayRough = str(datetime.datetime.now())
todayNum = int(dayRough[:4] + dayRough[5:7]+ dayRough[8:10])
todayNum
```

Out[1]:
```
20190509
```

In [2]:

```
# converts a number-date in the format YYYYMMDD to a normal Month day, year f
# input is an eight digit integer
# returns a date in the proper format
def numberToDate(num):
    # converts the eight digit number to a string
    strNum = str(num)
    # dictionary to easily create the month
    months = {'01':'January','02':'February','03':'March','04':'April','05':'
    # makes the first 4 digits (year) into a string
    year = strNum[:4]
    # pulls the month from the "months" dictionary using the next 2 digits
    month = months[strNum[4:6]]
    # converts the last 2 digits (day) to a string
    # making it an int first will get rid of a zero in a singe digit day
    day = str(int(strNum[6:]))
    # returns the string ex. June 7, 2018
    return str(month+' '+ day + ', '+ year)

# graphs the net Trump approval rating and the aggragated Trump score on the
# netDic is the dictionary, classified by number-date, of trumps approval rat
# score is the dictionary of aggregated Trump scores, adjusted for different
# titleName, xLab, yLab, and kType gives labels and types for the pandas.plot
# smooth is for some gradual exponential smoothing of the approval rating (ca
# adjust raises the approval rating graph to place the Trump score and approv
# final graph gives the graphed approvl rating and Trump score next to each o
# returns a pandas plot with the approval rating and Trump score on the same
def graphNetScore(netDic, score, approvalType = 'Net', titleName='',xLab=None
    approval = approvalType + '_Approval'
    # initiates the final dictionary for the approval rating
    netFin = {}
    # initiates a list for exponential smoothing
    numerList = []
    # loop runs through the approval rating dictionary and place them in a di
    for i in sorted(netDic.keys()):
        # since the score dictionary may have been assembled with a date rang
        if numberToDate(i) in list(score.keys()):
            # if you are attempting exponential smoothing
            if smooth != 0:
                # adds a number to the list of smooth raised to the index (th
                numerList.append(smooth**sorted(netDic.keys()).index(i))
                # puts the most current number added, smooth**sorted(netDic.k
                netFin[numberToDate(i)] = (netDic[i]*(smooth**sorted(netDic.k
            else:
                # without smoothing, simply adds the approval with the corres
                netFin[numberToDate(i)] = netDic[i]
    # initiates a dictionare containing date keys and values of dictionaries
    netWScore = {}
    # using the date keys, loops through the approval rating and Trump score
    for i in list(netFin.keys()):
        # puts the aprroval rating with a vertical shift and Trump score in t
        netWScore[i] = {approval:netFin[i]+adjust,'Trump_Score':score[i]}
    if scatter == True:
        return netWScore
    # creates a pandas dataframe with dates as keys,  and the approval rating
    netDfNew = pd.DataFrame(netWScore)
    netDf = netDfNew.transpose()
```

```
    # returns the DF on a plot, normally a line plot
    return netDf.plot(kind = kType, figsize = (12,8), title = titleName, x=xL

# aggregates the Trump score based on different parameters
# expon gives a value for exponential smoothing of the data, the smaller the
# numDateStart and numDateEnd gives a date range for data, with defaults as t
# house determines if the data should come from the Senate or House of Repres
# netDic gives the approval rating dictionary
# party determines if it is being aggregated by Republican (R), Democrat (D),
# congress is an integer, 115 or 116, to determine which session of congress
# initial is the dictionary where the data is pulled from, alway repInfo
# dayWeight is an integer determining if there is a moving time average, and
# onlyVote is a boolean which determines whether the data should include date
# returns the aggregated dictionary of Trump scores
def aggScore(expon = 0, numDateStart = 20170123, numDateEnd = todayNum, house
    # sets the session of Congress being scraped
    # if none, sets 117 for a range that will inclue 115 and 116
    if congress == None:
        conFirst = 115
        conSec = 117
    else:
        conFirst = congress
        conSec = congress + 1
    # initializes a dictionary based on numeric date to store the voting info
    dateVote = {}
    # loops through repInfo to extract each vote according to demographic
    for i in list(initial.keys()):
        # determines which house of congress if specified
        if house == None:
            # determines which party if specified
            if party == None:
                # loops through both sessions of Congress or just the one spe
                for z in range(conFirst,conSec):
                    # loops through the votes of each day to extract how many
                    for a in list(initial[i][z]['Votes'].keys()):
                        # determines if the date was already initialized in t
                        if a in list(dateVote.keys()):
                            # adds a vote to an existing dictionary value
                            dateVote[a] = dateVote[a] + initial[i][z]['Votes'
                        else:
                            # creates a new key with the vote as the value co
                            dateVote[a] = initial[i][z]['Votes'][a]
# coded the same with different initial values for party and house of Congres
            elif initial[i]['Party'] == party:
                for z in range(conFirst,conSec):
                    for a in list(initial[i][z]['Votes'].keys()):
                        if a in list(dateVote.keys()):
                            dateVote[a] = dateVote[a] + initial[i][z]['Votes'
                        else:
                            dateVote[a] = initial[i][z]['Votes'][a]
        elif initial[i]['House'] == house:
            if party == None:
                for z in range(conFirst,conSec):
                    for a in list(initial[i][z]['Votes'].keys()):
                        if a in list(dateVote.keys()):
                            dateVote[a] = dateVote[a] + initial[i][z]['Votes'
                        else:
```

```
                            dateVote[a] = initial[i][z]['Votes'][a]
                elif initial[i]['Party'] == party:
                    for z in range(conFirst,conSec):
                        for a in list(initial[i][z]['Votes'].keys()):
                            if a in list(dateVote.keys()):
                                dateVote[a] = dateVote[a] + initial[i][z]['Votes'
                            else:
                                dateVote[a] = initial[i][z]['Votes'][a]
# end of identical code
    # initializes the final dictionary
    scoreTrack = {}
    # initializes a score variable
    score = 0
    # determines if there is a time average argument (dayWeight) or exponenti
    # gives an aggregate score with no weighting for recency
    if (dayWeight == 0) & (expon == 0):
        # initializes yes variable for number of votes with
        yes = 0
        # variable for total votes
        total = 0
        # initializes variable to archive "total" for next loop
        totalPast = 0
        scoreList = []
        # loops through the dates in the approval rating dictionary that matc
        for i in sorted(netDic.keys()):
            # checks if there was a vote and if it is in the date range set
            if (i in list(dateVote.keys())) & ((i >= numDateStart) & (i <= nu
                # adds all the votes in a key to the total number of votes
                total += len(dateVote[i])
                # tallies yes votes
                for a in dateVote[i]:
                    if a == 'Yes':
                        yes += 1
                # gives a percentage of the score
                score = (yes/total)*100
                # rounds to two decimal places to prevent being overly precis
                if toDate == True:
                    scoreTrack[numberToDate(i)] = round(score,2)
                else:
                    scoreTrack[i] = round(score,2)
            # puts the last score value in if there was no vote on that day
            elif (score != 0) & (i <= numDateEnd):
                if toDate == True:
                    scoreTrack[numberToDate(i)] = round(score,2)
                else:
                    scoreTrack[i] = round(score,2)
    # loop does exponential weighting for the votes using a series based off
    # lower the number, the less recent votes are weighted, but this can be g
    elif expon != 0:
        # initializes list to store values for the exponentil average
        numeratorList = []
        # stores scores throughout to weight them differently each time
        scoreList = []
        # initializes the total variable for votes
        total = 0
        # counter for weighting the votes
        counter = 0
```

```
        for i in range(len(sorted(netDic.keys()))):
            # makes sure vote is during the Presidency and is within the date
            if (sorted(netDic.keys())[i] in list(dateVote.keys()))&((sorted(n
                # adds one to the counter
                counter+=1
                # raises the number for exponential weighting to the counter
                numeratorList.append(expon**(counter))
                if sorted(netDic.keys())[i] in list(dateVote.keys()):
                    # intializes the yesCount for that day
                    yesCount = 0
                    # gives the total number of votes for that day
                    total = len(dateVote[sorted(netDic.keys())[i]])
                    # loops to get yes votes
                    for b in dateVote[sorted(netDic.keys())[i]]:
                        if b == 'Yes':
                            # adds one for every 'Yes'
                            yesCount+=1
                    # gives score for the day
                    score = (yesCount/total)*100
                    # adds the score to the score list
                    scoreList.append(score)
                # initializes the final weighted score for the day
                scoreFin = 0
                # loops through score list to weight each one
                for a in range(len(scoreList)):
                    # puts the exponent raised to the power of which day it w
                    # divides it by the total of the list
                    # will these fractions would end up equalling one summed
                    scoreFin += ((expon**(a+1))/sum(numeratorList))*scoreList
            # adds the last scoreFin to the list if there were no votes that
            if total != 0:
                if toDate == True:
                    scoreTrack[numberToDate(sorted(netDic.keys())[i])] = roun
                else:
                    scoreTrack[sorted(netDic.keys())[i]] = round(scoreFin,2)
    # does the moving time average based on dayWeight
    else:
        # initiates dailyScore dictionary to store yes votes in the period an
        dailyScore = {}
        # initiates the total votes variable for that day
        totalDay = 0
        # loops through the dates based on an index
        for i in range(len(sorted(netDic.keys()))):
            # checks if they are during the presidency and within the date ra
            if (sorted(netDic.keys())[i] in list(dateVote.keys())) & ((sorted
                # initiates the yes variable for each day
                yesCount = 0
                # counts the votes with the Trump administration
                for b in dateVote[sorted(netDic.keys())[i]]:
                    if b == 'Yes':
                        yesCount+=1
                # stores votes with and total votes for each day
                dailyScore[sorted(netDic.keys())[i]] = {'Yes':yesCount, 'Numb
                # initiates the total votes with and total votes variables
                yes = 0
                total = 0
                # aggregates for each day that hasn't hit the time frame for
```

```
                if i < dayWeight:
                    # loops through every day up to then
                    for c in sorted(netDic.keys())[:i+1]:
                        # totals yes votes and the total votes
                        if c in list(dateVote.keys()):
                            yes += dailyScore[c]['Yes']
                            total += dailyScore[c]['Number']
                # loops through the number of days before that day that are i
                else:
                    for c in sorted(netDic.keys())[i-dayWeight:i+1]:
                        if c in list(dailyScore.keys()):
                            yes += dailyScore[c]['Yes']
                            total += dailyScore[c]['Number']
                # stores the total votes with and total votes cast
                yesDay = dailyScore[c]['Yes']
                totalDay = dailyScore[c]['Number']
                # gets the Trump score, but doesnt include days where no vote
                if total != 0 & (not ((totalDay != 0) & (onlyVote == True))):
                    score = (yes/total)*100
                    if toDate == True:
                        scoreTrack[numberToDate(sorted(netDic.keys())[i])] =
                    else:
                        scoreTrack[sorted(netDic.keys())[i]] = round(score,2)
            # puts the last vaue in for days where no votes are cast
            elif (score != 0) & ((sorted(netDic.keys())[i] <= numDateEnd)& (n
                if toDate == True:
                    scoreTrack[numberToDate(sorted(netDic.keys())[i])] = roun
                else:
                    scoreTrack[sorted(netDic.keys())[i]] = round(score,2)
    # returns the dictionary with dates as keys and the Trump score aggregate
    return scoreTrack
# extracts mean, median, and mode of a numeric list
# newList is a list of numeric values
# returns a dictionary with the mean, median, and mode
def stats(newList):
    # sums the mean and places it over the number of values in the list
    mean = round(sum(newList)/len(newList),2)
    # finds the center value or values
    # even number of values
    if (len(newList) % 2) == 0:
        ind = int(len(newList) / 2)
        # avergaes the two middle values
        median = (newList[ind-1]+newList[ind])/2
    # finds the middle number for an odd number of values
    else:
        ind = int((len(newList)//2)+1)
        median = newList[ind-1]
    # removes all repeat values for a new list
    elements = list(set(newList))
    # initiates variable to count the number of times a number appears
    topNumFirst = 0
    for i in elements:
        # counts the number of times the value appears
        topNum = newList.count(i)
        # replaces the value if greater than before
        if topNum > topNumFirst:
            # puts the number in a list
```

```
            top = [i]
            topNumFirst = topNum
        # adds to the list if there are values that appear the same number of
        elif topNum == topNumFirst:
            top = top + [i]
    # makes the list into the mode variable
    mode = top
    # places the mean, median, and mode in a dictionary to return
    return {'Mean': mean, 'Median': median, 'Mode': mode}
# returns the mean of the Trump approval over a certain period of time
# midDate is a numeric date to determine the middle point with times on eithe
# appDic determines if you want net approval, approval, or disapproval
# days gives the time range to calculate the average
# returns a dcitionary witht the average before and after midDate
def timeApproval(midDate, appDic = net, days = 14):
    # finds the index in the approval dictionary
    midInd = list(appDic.keys()).index(midDate)
    # finds the start date
    startInd = midInd-days
    #finds the end date
    endInd = midInd + days
    # using the stats function, the mean over the time interval is returned i
    return {'Before_Mean' : stats(list(appDic.values())[startInd:midInd+1])['
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-d844b5216f58> in <module>
     69 # onlyVote is a boolean which determines whether the data should in
clude dates where there were not votes
     70 # returns the aggregated dictionary of Trump scores
---> 71 def aggScore(expon = 0, numDateStart = 20170123, numDateEnd = today
Num, house = None,netDic = net, party=None, congress=None, initial = repInf
o, dayWeight = 0, onlyVote = False, toDate = True):
     72     # sets the session of Congress being scraped
     73     # if none, sets 117 for a range that will inclue 115 and 116

NameError: name 'net' is not defined
```

President Trump's Net-approval Rating and how Congress Votes with Him

Approval Rating

The approval rating of a President is impacted by many things. Their recent public statements, scandals, economic factors out of their control, economic policy, and even whether or not a President can relate to the average voter affects their approval rating. I want to see if the way Congress votes with President Trump affects his approval rating, even during his tenure with investigations, scandals, and controversial statements that seem to come at least once a week. The President's approval rating also goes up and down frequently, but the range of this shift is currently only within about ten percentage points.

President Trump's approval and disapproval rating move in a similar fashion, but not identically. For this reason, I used difference between the two, net-approval, for my testing.

Five Thirty Eight

Five Thirty Eight is a data analysis site that uses statistical models and aggregations of data to inform the average person about trends. Their main focus is politics, but they have expanded into sports forecasting for college and professional sports. I used their data for the approval ratings of Donald Trump because they weight, correct for partisanship, and average polls for every day. I believe this statistical analysis makes it the most reliable source for the President's approval and disapproval ratings.

Trump Score

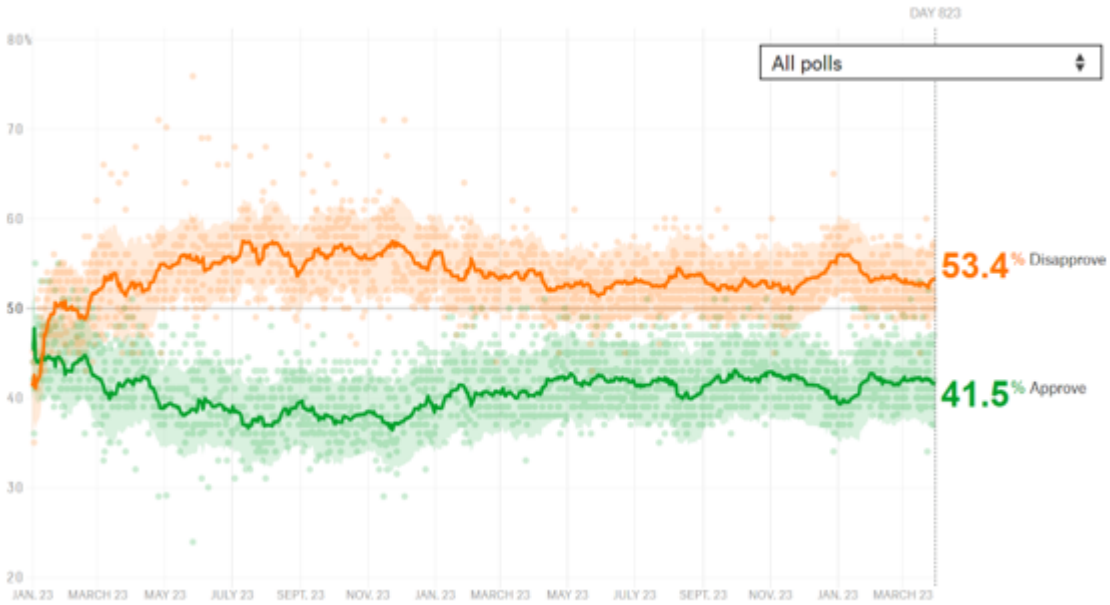
The Trump Score is a term that Five Thirty Eight uses to describe the percent frequency that a Senator or Representative votes with President Trump. Using their data on individual Trump Scores, I plan to aggregate it to track the overall Trump Score compared to the President's approval rating over time.

Scraping the Approval and Dsiapproval Ratings

This process is relatively easy, since Five Thirty Eight has a graphic on their page operated with javascript. I managed to scrape this as a massive text string, then using keywords in the script, the number that

In [78]:
```
from IPython.display import Image
Image("AppGraph.png")
```

Out[78]:



In [2]:

```
import requests
from bs4 import BeautifulSoup
url = 'https://projects.fivethirtyeight.com/trump-approval-ratings/?ex_cid=rr
website = requests.get(url)
soup = BeautifulSoup(website.content, 'html.parser')
# scrapes the graph on fivethirtyeight.com to find the approval ratings
numbers = soup.find_all('script', type ='text/javascript')[2].get_text().spli
# initiates three dictionaries for approval, disapproval, and net-approval ra
app = {}
dis = {}
net = {}
# loop runs through the javascript to extract the necessary numbers
for i in numbers[1].split('},{'):
    new = i.lstrip("['{")
    newList = new.split(',')
    # cuts the values that are projected in the javascript
    if newList[1] == '"future":false':
        # makes the date into an integer
        date = int(newList[0][8:12]+newList[0][13:15]+newList[0][16:18])
        appList = newList[3].split('"')
        appNum = float(appList[-2])
        disList = newList[6].split('"')
        disNum = float(disList[-2])
        app[date] = appNum
        dis[date] = disNum
        #subtracts disapproval from approval for net approval
        net[date] = appNum - disNum
net
```

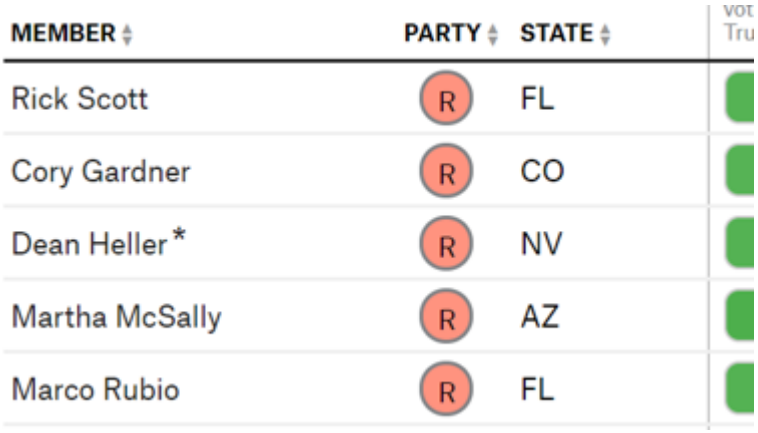
...

Extracting Links and Information

On the initial page for https://projects.fivethirtyeight.com/congress-trump-score/ (https://projects.fivethirtyeight.com/congress-trump-score/), there is a table with each Senator's and Representative's Trump Score, state, district if they are a Representative, and party. Embedded in the name is a hyperlink to each person's page which outlines their individual votes compared to the Trump Administration's public stance. Although it does not specify whether they are a Senator or Representative when scraping the site, this can be determined if they only have a State listed (Senator) or have a State and Congressional District (Representative). Finally, the party can be extracted from a text string with either an R (Republican), D (Democrat), or I (Independent). This information is stored in separate lists to be aggregated later.

In [79]:
```
Image('firstPage.png')
```

Out[79]:



MEMBER	PARTY	STATE	Vot Tru
Rick Scott	R	FL	
Cory Gardner	R	CO	
Dean Heller *	R	NV	
Martha McSally	R	AZ	
Marco Rubio	R	FL	

In [77]:

```
url = 'https://projects.fivethirtyeight.com/congress-trump-score/'
website = requests.get(url)
soup = BeautifulSoup(website.content, 'html.parser')
# finds all the links for congressional representatives
senateLinks = soup.find_all('td', class_='name long')
# pulls the party from each rep
partyEl = soup.find_all('td', class_='party')
# initiates the respective dictionaries for links and parties
repDic = {}
partyDic = {}
# scrapes the link for each representative
for i in senateLinks:
    link = i.find('a').attrs['href']
    rep = i.get_text().replace(' ','_')
    repFin = rep.replace('*','')
    repDic[repFin] = link
# scrapes to find the party of each representative
for a in range(len(partyEl)):
    party = partyEl[a].get_text().strip()
    repName = list(repDic.keys())[a]
    partyDic[repName] = party
repDic
```

Out[77]:

```
{'Rick_Scott': '/congress-trump-score/rick-scott/',
 'Cory_Gardner': '/congress-trump-score/cory-gardner/',
 'Dean_Heller': '/congress-trump-score/dean-heller/',
 'Martha_McSally': '/congress-trump-score/martha-mcsally/',
 'Marco_Rubio': '/congress-trump-score/marco-rubio/',
 'Ron_Johnson': '/congress-trump-score/ron-johnson/',
 'Patrick_J._Toomey': '/congress-trump-score/patrick-j-toomey/',
 'Thom_Tillis': '/congress-trump-score/thom-tillis/',
 'David_Perdue': '/congress-trump-score/david-perdue/',
 'Richard_Burr': '/congress-trump-score/richard-burr/',
 'Johnny_Isakson': '/congress-trump-score/johnny-isakson/',
 'John_Cornyn': '/congress-trump-score/john-cornyn/',
 'Susan_M._Collins': '/congress-trump-score/susan-m-collins/',
 'Rob_Portman': '/congress-trump-score/rob-portman/',
 'Ted_Cruz': '/congress-trump-score/ted-cruz/',
 'John_McCain': '/congress-trump-score/john-mccain/',
 'Joni_Ernst': '/congress-trump-score/joni-ernst/',
 'Kyrsten_Sinema': '/congress-trump-score/kyrsten-sinema/',
 'Jeff_Flake': '/congress-trump-score/jeff-flake/',
```

In [4]:

```
# uses the state listing to find whether they are a Senator or Representative
stateList = soup.find_all('td', class_='state')
stateListText = []
# cleans each state label down
for i in stateList:
    state = i.get_text().strip()
    stateListText.append(state)
#initiates lists for the Senate and House
senate = []
house = []
for a in stateListText:
    # if the length of the label is longer than 2, then the district is inclu
    if len(a) > 2:
        house.append(a)
    # otherwise puts the namein the Senate list
    else:
        senate.append(a)
# creates final lists for the House and Senate
senators = list(partyDic.keys())[:len(senate)]
representatives = list(partyDic.keys())[len(senate):]
senators[:10]
```

Out[4]:

```
['Rick_Scott',
 'Cory_Gardner',
 'Dean_Heller',
 'Martha_McSally',
 'Marco_Rubio',
 'Ron_Johnson',
 'Patrick_J._Toomey',
 'Thom_Tillis',
 'David_Perdue',
 'Richard_Burr']
```

In [5]:

```
# dictionary to turn months into numbers
months = {'Jan.':'01', 'Feb.':'02',  'March':'03', 'April':'04', 'May':'05',
# initializes dictionary for storing all the information of each voting membe
repInfo = {}
# goes through each url to extract specific voting information from each memb
for j in list(repDic.keys()):
    # builds the url for beautiful soup
    url = 'https://projects.fivethirtyeight.com' + repDic[j]
    website = requests.get(url)
    soup = BeautifulSoup(website.content, 'html.parser')
    tables = soup.find_all('table', class_='member-votes')
    dateList = []
    voteList= []
    # finds the votes and dates for each vote
    for i in tables:
        dateList = dateList + i.find_all('td', class_='date')
        voteList = voteList + i.find_all('span', class_='value')
    # list stores yes (with Trump) and no
    newVote = []
    for i in voteList:
        # indicated on the website with an image of a check or x
        if i.find('img') != None:
            vote = i.find('img').attrs['src']
            # uses last digit of the file that stores checks and x's to deter
            if vote[-1] == '1':
                newVote.append('Yes')
            else:
                newVote.append('No')
    # stores the dates of each vote in a list
    dates = []
    # creates an integer date
    for i in dateList:
        date = i.get_text()
        dateSplit = date.split()
        # creates the numeric year, and puts in 2019 if blank
        if len(dateSplit) == 2:
            year = '2019'
            dateSplit[1] = dateSplit[1] + ','
        else:
            year = dateSplit[2]
        # creates a day, if single digit, with a zero in front of it
        if len(dateSplit[1]) == 2:
            day = '0' + dateSplit[1].strip(',')
        else:
            day = dateSplit[1].strip(',')
        # extracts the month from the month dictionary for the numeric charac
        month = months[dateSplit[0]]
        # puts the strings together and turns them into an integer
        finalDate = int(year+month+day)
        dates.append(finalDate)
    # initializes dctionaries for each session of congress along with variabl
    votes116 = {}
    total116 = 0
    votes115 = {}
    total115 = 0
    # runs through the dates
```

```
    for i in range(len(newVote)):
        # if after January 3, 2019, in 116th Congress
        if dates[i] >= 20190103:
            # adds to the total votes for that day for the representative
            total116 += 1
            # places each day's votes in a list for 116th congress
            if dates[i] == dates[i-1]:
                votes116[dates[i]] = [newVote[i-1],newVote[i]]
            else:
                votes116[dates[i]] = [newVote[i]]
        # all others go in the 115th Congress
        else:
            total115 += 1
            if dates[i] == dates[i-1]:
                votes115[dates[i]] = [newVote[i-1],newVote[i]]
            else:
                votes115[dates[i]] = [newVote[i]]
    # places the individual votes and the total number in each representative
    repInfo[j] = {115:{'Votes':votes115,'Total':total115}, 116:{'Votes':votes
    # puts the house of congress for each representative in the dictionary
    if j in senators:
        repInfo[j]['House'] = 'Senate'
    else:
        repInfo[j]['House'] = 'Representatives'
    repInfo[j]['Party'] = partyDic[j]
```

Information for Each Congressman

This dictionary is used throughout, with arguments that can filter people with specific information and dates before or after different dates. The session of Congress can be filtered out for each vote as well.

{'Rick_Scott': {115: {'Votes': {}, 'Total': 0},

116: {'Votes': {20190411: ['Yes'],

```
    20190314: ['Yes'],

    20190313: ['Yes'],

    20190228: ['Yes'],

    20190225: ['Yes'],

    20190214: ['Yes'],

    20190204: ['No'],

    20190124: ['Yes', 'Yes'],

    20190117: ['Yes'],

    20190116: ['Yes']},
```

'Total': 11},

'House': 'Senate',

'Party': 'R'}

Every Senator and Representative has their information formatted in this way. A 'Yes' means the vote was with the Trump Administration, and a 'No' means against. These are in lists for uniform coding throughout the aggregating function that also accounts for multiple votes on one day.

In [80]:
```
Image('links.png')
```

Out[80]:

DATE	MEASURE	TRUMP POSITION	SCOTT VOTE	AGREE WITH TRUMP?
April 11	Nomination of David Bernhardt to be secretary of the interior (56-41)	Support	Yes	✔
March 14	Overturning President Trump's emergency declaration for border wall funding (59-41)	Oppose	No	✔
March 13	Ending U.S. military assistance to Saudi-led forces in the war in Yemen (54-46)	Oppose	No	✔
Feb. 28	Nomination of Andrew Wheeler to be administrator of the Environmental Protection Agency (52-47)	Support	Yes	✔

In [6]:
```
repInfo
```

. . .

Aggregating the Trump Score

The function for aggregating the Trump Score has up to 11 arguments. They are not necessarily meant to be used at the same time, but can be used for different weighting and filtering. The below dictionary is the base function, with dates as the keys and the overall average Trump Score up to that date as the values.

In [81]:
```
aggScore()
```

. . .

Graphs Comparing Trump Score to Net-approval

I wanted to see a good visual comparing the Trump Score to net-approval, but Congress is somewhat volatile while approval ratings move up and down relatively slowly. Using my function for aggregating the Trump Score and the function for graphing it next to the net-approval rating, I was able to create many visuals to test correlation.

One capability the program has, but I did not use due to an assumption that net-approval has better correlation than approval or disapproval, is to graph the Trump Score next to the approval rating and disapproval rating.

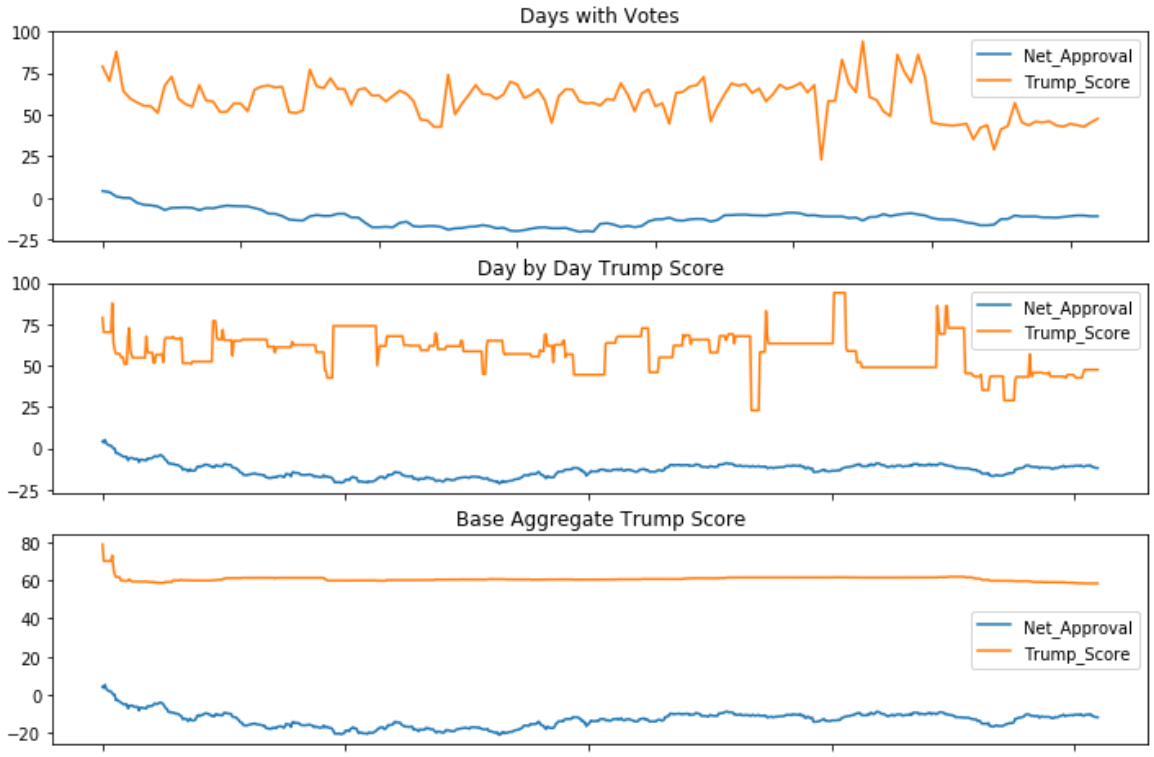
Initial Graphs

These three graphs show the fitting of the data.

1. The first graph draws lines between voting days with no weighting.
2. The second holds constant the last vote until a new vote, again with no weighting.
3. The third gives a straight aggregate Trump Score, taking a new avaerage every day, smoothing out a ton over time.

In [30]:
```
agg = aggScore(dayWeight = 1)
I = aggScore()
onlyVoteDic = aggScore(dayWeight = 1,onlyVote = True)
figDem = plt.figure()
onlyVote = figDem.add_subplot(311)
dayBDay = figDem.add_subplot(312)
aggGraph = figDem.add_subplot(313)
graphNetScore(net, onlyVoteDic, titleName = 'Days with Votes', axis = onlyVot
graphNetScore(net, agg, titleName = 'Day by Day Trump Score', axis = dayBDay)
graphNetScore(net, I, titleName = 'Base Aggregate Trump Score', axis = aggGra
```

Out[30]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x24ddf130588>
```



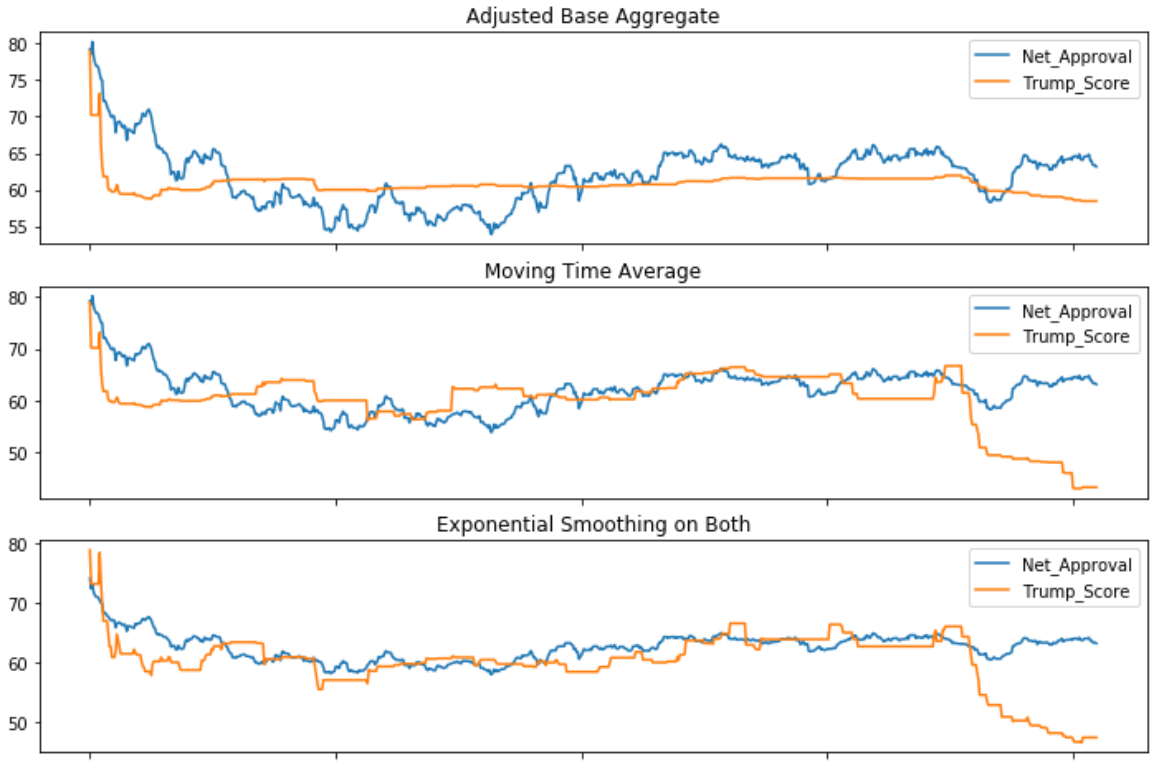
Comparisons and Fitting

These graphs show the Net-approval rating next to the Trump Score by shifting the graph upward using the adjust variable in the graphNetScore function.

1. The first graph is still just the aggregate Trump Score.
    - This graph smooths very quickly, and the only real inflection point is when the 116th Congress comes in.
2. The second displays a moving time average for all of Congress with a time of 100 days.
3. The third graph shows an exponential average where the most recent votes get a higher weight, but all votes get considered.

In [9]:
```
figTime = plt.figure()
adjust = figTime.add_subplot(311)
moving = figTime.add_subplot(312)
exponent = figTime.add_subplot(313)
graphNetScore(net, I, titleName = 'Adjusted Base Aggregate', adjust = 75, axi
exp = aggScore(expon = 1.09)
graphNetScore(net, exp, titleName = 'Exponential Smoothing on Both', smooth =
weighted = aggScore(dayWeight = 100)
graphNetScore(net, weighted, titleName = 'Moving Time Average', adjust = 75,
```

Out[9]: `<matplotlib.axes._subplots.AxesSubplot at 0x24dde881a90>`

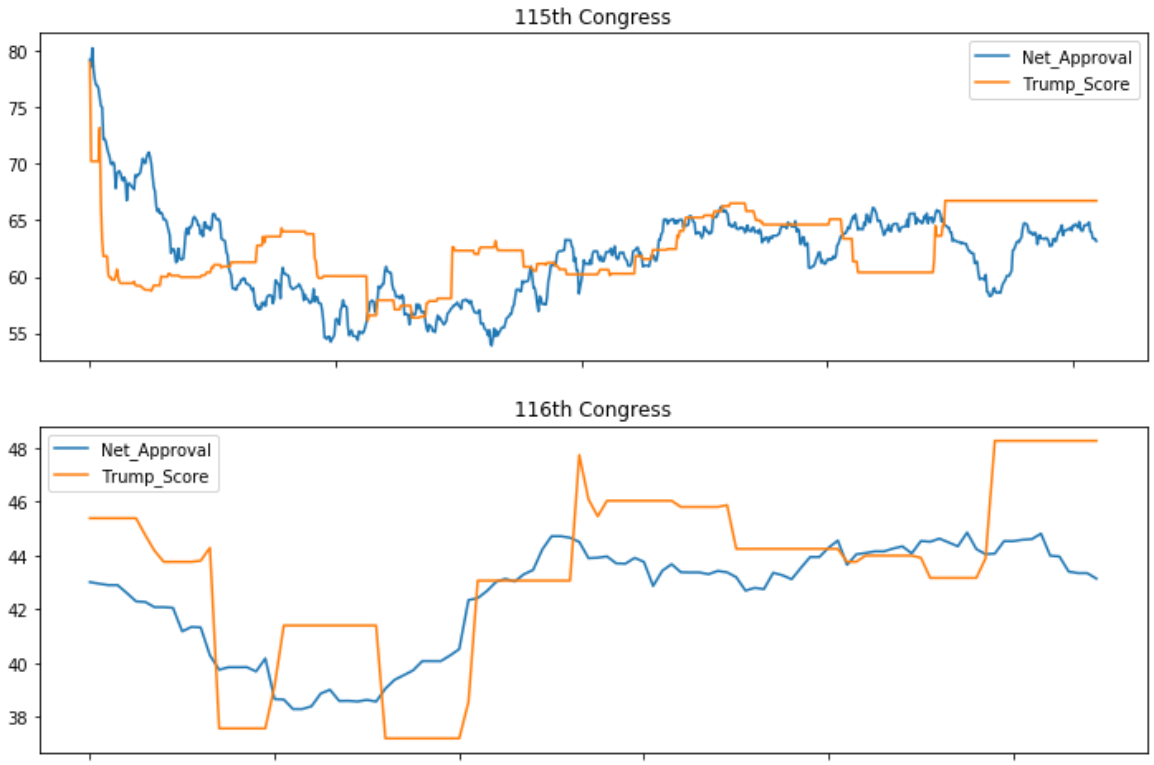


Congresses

The 115th Congress and 116th Congress show some correlation with President Trump's net-approval rating. One thing to notice though is the vertical adjustment. For the 115th Congress, controlled by Republicans, the net-approval rating must be adjusted upwards by 75. For the 116th Congress, with a Democratic controlled House of Representatives, the vertical shift is only 55. Also, the larger period for the 115th Congress works well with a moving time average of 100 days. The 116th Congress uses an exponential average, since the 116th Congress has not been serving a significant amount of time.

In [10]:
```
figCon = plt.figure()
plt115 = figCon.add_subplot(211)
plt116 = figCon.add_subplot(212)
con115 = aggScore(dayWeight = 100, congress = 115)
graphNetScore(net, con115, titleName = '115th Congress', adjust = 75, axis =
con116 = aggScore(expon = 1.5, congress = 116)
graphNetScore(net, con116, titleName = '116th Congress', adjust = 55, axis =
```

Out[10]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x24dde325c88>
```

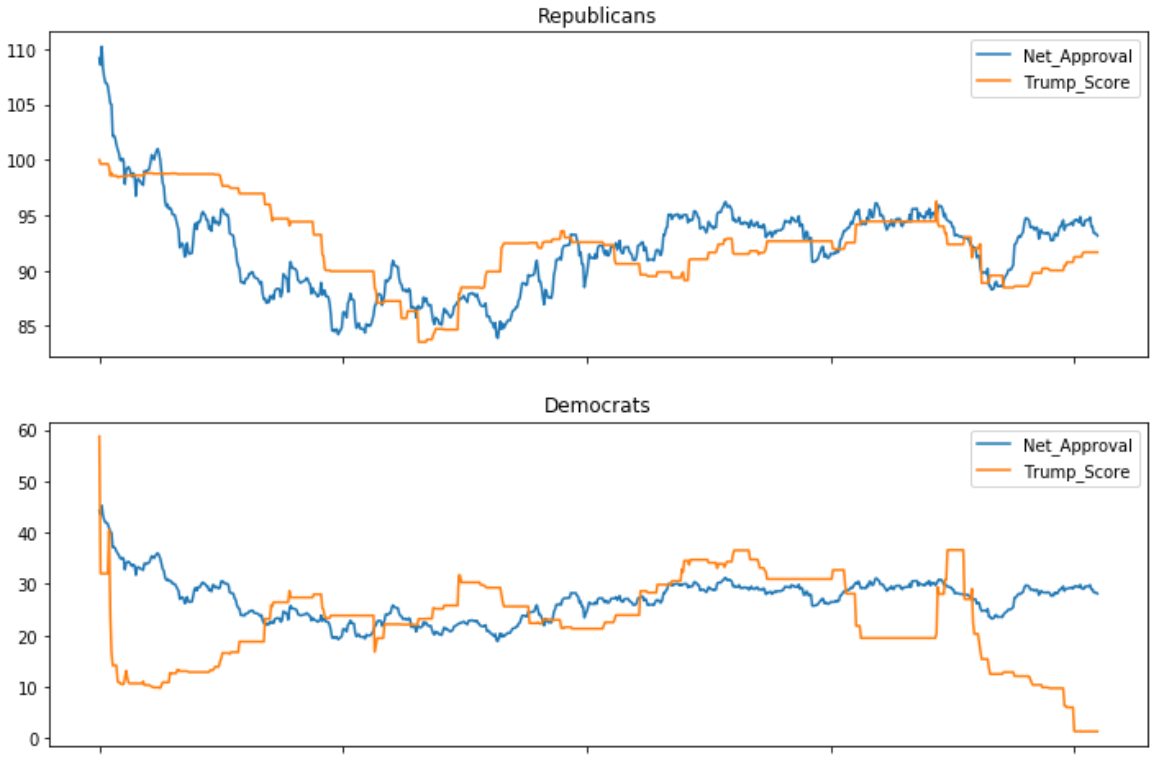


Parties

I was interested to see how President Trump's net-approval compared with the way the parties vote. I used the moving time avaerage with a 100 day range for each graph. Also notice the Republican graph has a vertical shift of 105, while the Democratic graph has a shift of 40. The Republican graph shows a decent correlation, with similar shifts, but the Trump Score is more volatile than the net-approval rating.

In [12]:
```
figParty = plt.figure()
repPlt = figParty.add_subplot(211)
demPlt = figParty.add_subplot(212)
repub = aggScore(party = 'R', dayWeight = 100)
graphNetScore(net, repub, titleName = 'Republicans', adjust = 105, axis = rep
dem = aggScore(party = 'D', dayWeight = 100)
graphNetScore(net, dem, titleName = 'Democrats', adjust = 40, axis = demPlt)
```

Out[12]: <matplotlib.axes._subplots.AxesSubplot at 0x24dde5d4d30>



Congresses Split by Party

Other than Republican 115th, none of the graphs show a very strong correlation. These are the important aspects of the graphs:

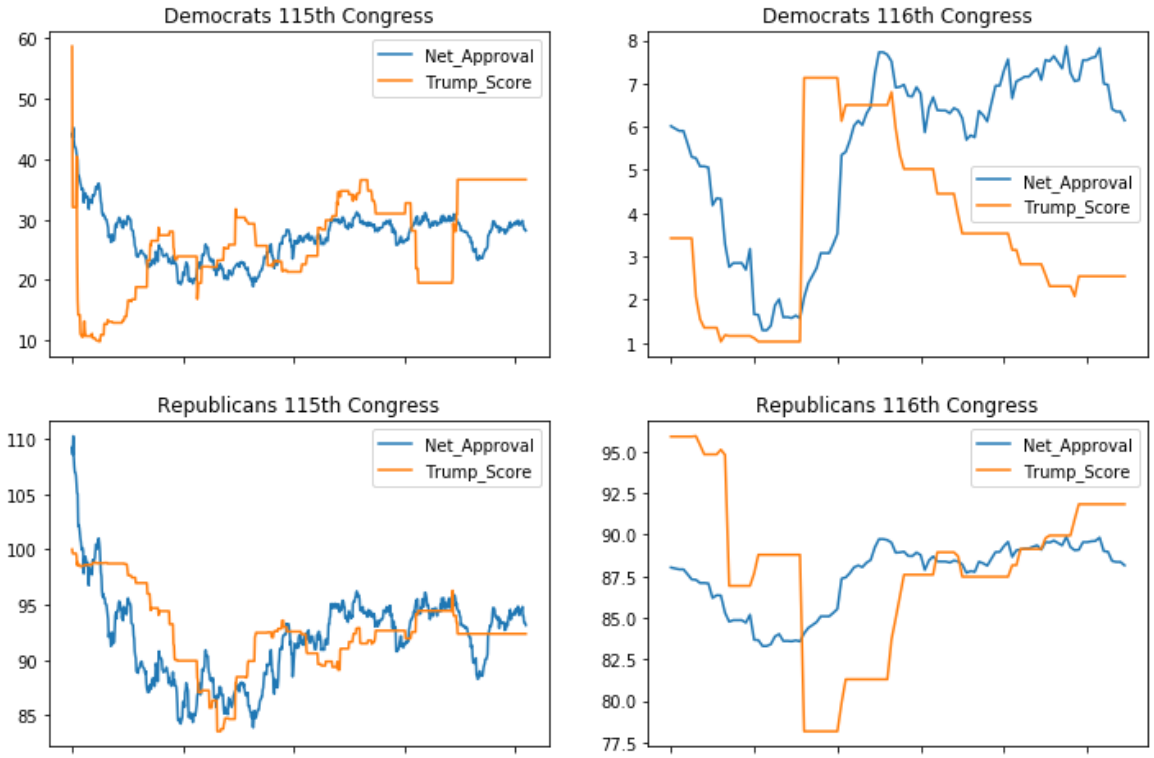
1. Democrats 115
   - Moving time average of 100 days
   - Vertical shift of 40
2. Democrats 116
   - Exponential average with a constant of 1.1
   - Vertical shift of 18
3. Republicans 115
   - Moving time average of 100 days
   - Vertical shift of 105
4. Republicans 116
   - Exponential average with a constant of 1.1

- Vertical shift of 100

Something important to notice is the method of aggregating the Trump Score used. Since the 116th Congress is relatively new, there is not a large enough period to use the moving time average effectively. Also, the vertical shift for Democrats is expected to be lower, but it even drops 22 points from the 115th to 116th Congress due to controlling the House of Representatives. This means Democratic legislation that President Trump opposes reaches the floor more often.

In [13]:
```
figPartCon = plt.figure()
dem115 = figPartCon.add_subplot(221)
dem116 = figPartCon.add_subplot(222)
rep115 = figPartCon.add_subplot(223)
rep116 = figPartCon.add_subplot(224)
demCon115 = aggScore(dayWeight = 100, party = 'D', congress = 115)
demCon116 = aggScore(expon = 1.1, party = 'D', congress = 116)
repCon115 = aggScore(dayWeight = 100, party = 'R', congress = 115)
repCon116 = aggScore(expon = 1.1, party = 'R', congress = 116)
graphNetScore(net, demCon115, titleName = 'Democrats 115th Congress', adjust
graphNetScore(net, demCon116, titleName = 'Democrats 116th Congress', adjust
graphNetScore(net, repCon115, titleName = 'Republicans 115th Congress', adjus
graphNetScore(net, repCon116, titleName = 'Republicans 116th Congress', adjus
```

Out[13]: <matplotlib.axes._subplots.AxesSubplot at 0x24dde4dbe10>



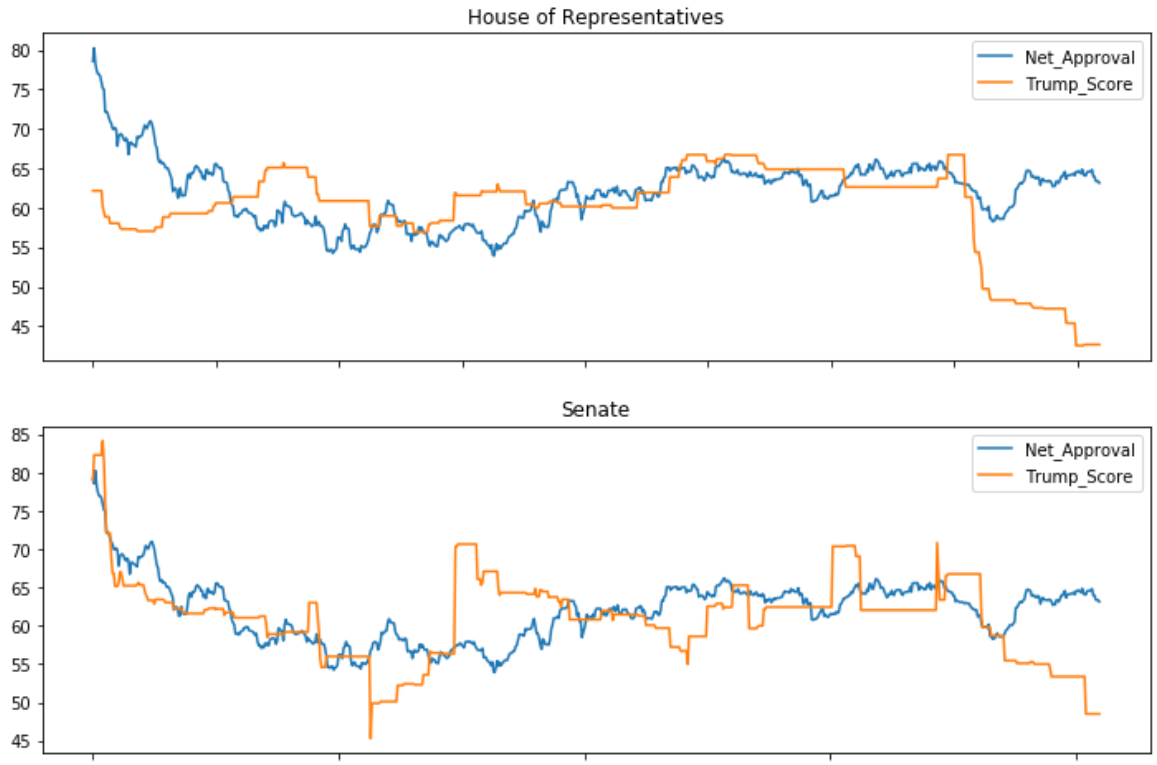
House of Representatives and Senate

Neither of these graphs show a strong correlation between the Trump Score and net-approval. They are both done with a moving time average of 100 days and a vertical shift of 75. One thing to notice is that even though the Senate actually gained a few seats in the 116th Congress, there is

still an inflection point downwards where the 116th Congress starts. This is probably due to defections from Republicans on votes during the government shutdown and some foreign policy votes.

In [14]:
```
figHouse = plt.figure()
housePlt = figHouse.add_subplot(211)
senPlt = figHouse.add_subplot(212)
houseData = aggScore(house = 'Representatives', dayWeight = 100)
senateData = aggScore(house = 'Senate', dayWeight = 100)
graphNetScore(net, houseData, titleName = 'House of Representatives', adjust
graphNetScore(net, senateData, titleName = 'Senate', adjust = 75, axis = senP
```

Out[14]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x24dde7918d0>
```

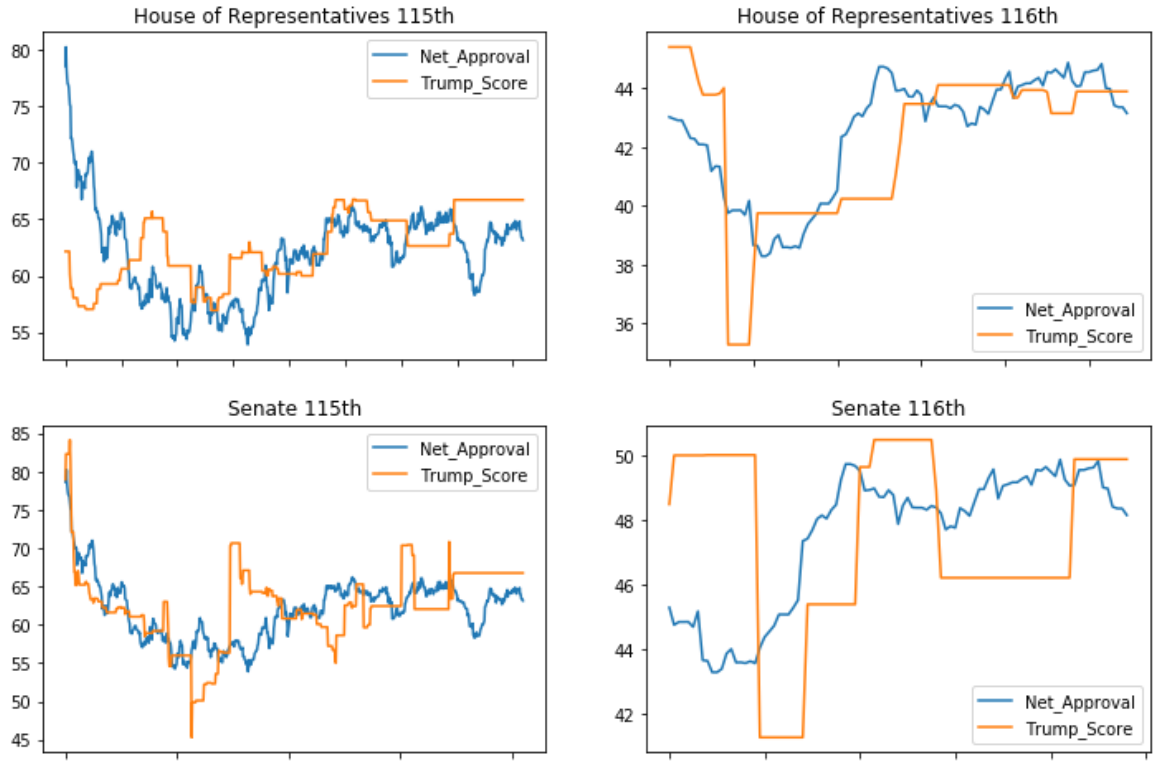


House and Congress

These graphs look at the correlation with respect to the House of Representatives and Senate. The adjustment number drops for both the House and Senate during the 116th Congress. The strongest correlation is in the House of Representatives during the 116th Congress. However, the Trump Score during this period had a significant drop to around 43 from the 115th Congress, where the House consistently had a Trump Score near 60.

In [15]:
```
figHousePart = plt.figure()
house115Plt = figHousePart.add_subplot(221)
house116Plt = figHousePart.add_subplot(222)
sen115Plt = figHousePart.add_subplot(223)
sen116Plt = figHousePart.add_subplot(224)
house115Data = aggScore(house = 'Representatives', congress = 115, dayWeight
house116Data = aggScore(house = 'Representatives', congress = 116, expon = 1.
senate115Data = aggScore(house = 'Senate', congress = 115, dayWeight = 100)
senate116Data = aggScore(house = 'Senate', congress = 116, expon =1.5)
graphNetScore(net, house115Data, titleName = 'House of Representatives 115th'
graphNetScore(net, house116Data, titleName = 'House of Representatives 116th'
graphNetScore(net, senate115Data, titleName = 'Senate 115th', axis = sen115Pl
graphNetScore(net, senate116Data, titleName = 'Senate 116th', axis = sen116Pl
```

Out[15]: <matplotlib.axes._subplots.AxesSubplot at 0x24dde480ba8>



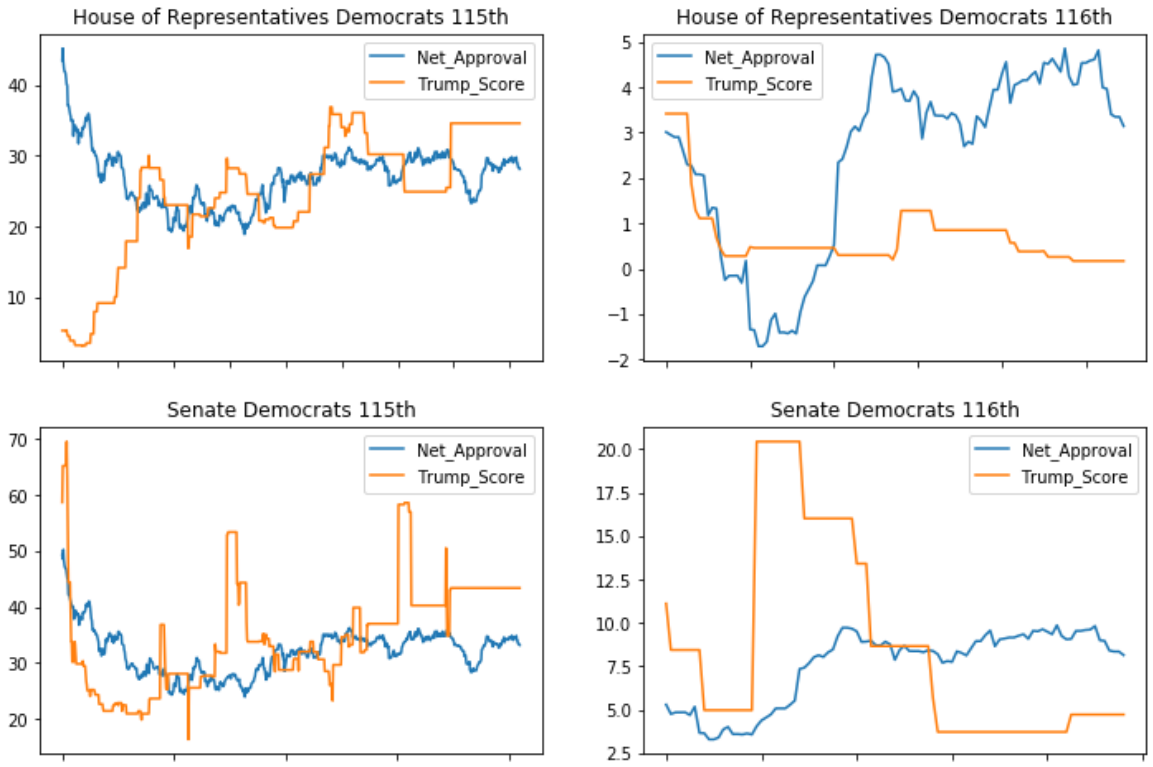
Parties Split by House and Congress

Of the next 8 graphs, none show a very strong correlation, but the best correlations come from the Republican members of the House of Representatives during both the 115th and 116th Congress. This could explain why the House consistently has a better correlation even when Democrats have

control. Republicans vote with a similar base as Donald Trump, so when that base is unhappy, they are more likely to vote against the President, but Democrats vote with an opposing base, making them far more likely to vote against the President.

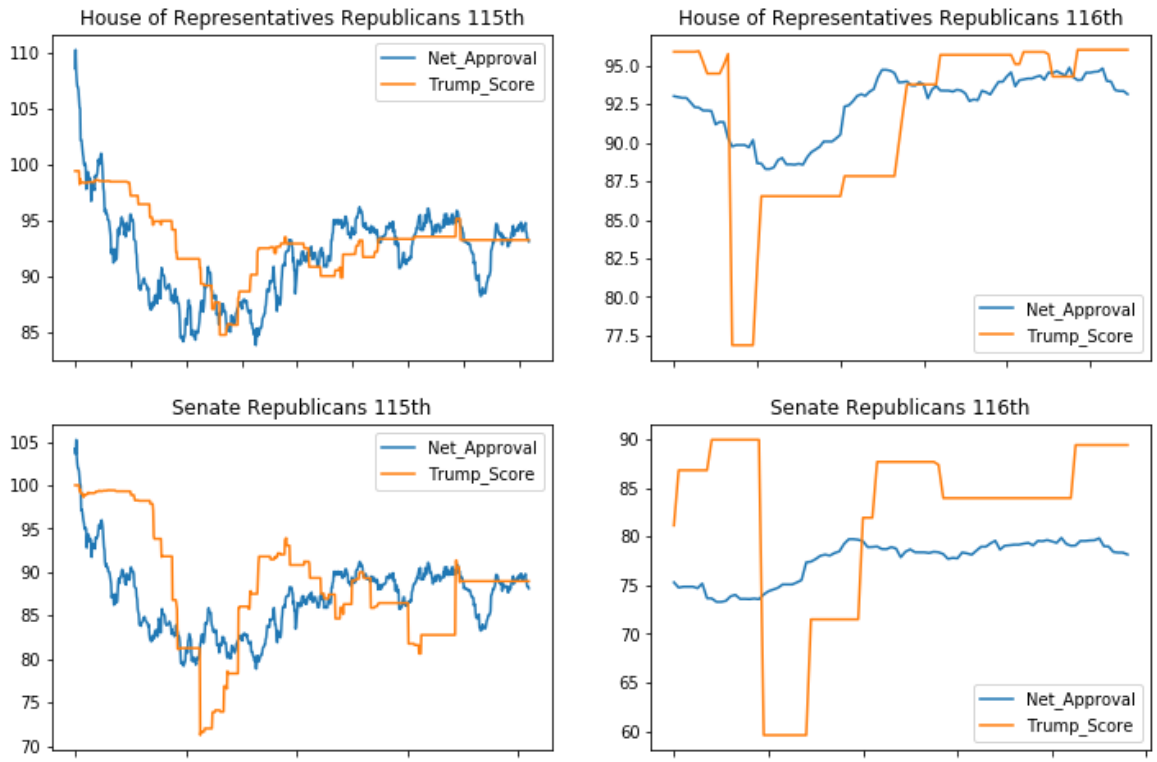
In [16]:
```
figHousePartCon = plt.figure()
house115DemPlt = figHousePartCon.add_subplot(221)
house116DemPlt = figHousePartCon.add_subplot(222)
sen115DemPlt = figHousePartCon.add_subplot(223)
sen116DemPlt = figHousePartCon.add_subplot(224)
house115DemData = aggScore(house = 'Representatives', congress = 115, party =
house116DemData = aggScore(house = 'Representatives', congress = 116, party =
senate115DemData = aggScore(house = 'Senate', congress = 115, party = 'D', da
senate116DemData = aggScore(house = 'Senate', congress = 116, party = 'D', ex
graphNetScore(net, house115DemData, titleName = 'House of Representatives Dem
graphNetScore(net, house116DemData, titleName = 'House of Representatives Dem
graphNetScore(net, senate115DemData, titleName = 'Senate Democrats 115th', ax
graphNetScore(net, senate116DemData, titleName = 'Senate Democrats 116th', ax
```

Out[16]: <matplotlib.axes._subplots.AxesSubplot at 0x24dde185c18>



In [17]:
```
figHouseRepCon = plt.figure()
house115RepPlt = figHouseRepCon.add_subplot(221)
house116RepPlt = figHouseRepCon.add_subplot(222)
sen115RepPlt = figHouseRepCon.add_subplot(223)
sen116RepPlt = figHouseRepCon.add_subplot(224)
house115RepData = aggScore(house = 'Representatives', congress = 115, party =
house116RepData = aggScore(house = 'Representatives', congress = 116, party =
senate115RepData = aggScore(house = 'Senate', congress = 115, party = 'R', da
senate116RepData = aggScore(house = 'Senate', congress = 116, party = 'R', ex
graphNetScore(net, house115RepData, titleName = 'House of Representatives Rep
graphNetScore(net, house116RepData, titleName = 'House of Representatives Rep
graphNetScore(net, senate115RepData, titleName = 'Senate Republicans 115th',
graphNetScore(net, senate116RepData, titleName = 'Senate Republicans 116th',
```

Out[17]: <matplotlib.axes._subplots.AxesSubplot at 0x24dde98c048>



Difference of Mean for Net-approval Compared to a One Vote Trump Score

To see if the President's net-approval rating is affected by certain congressional votes, I compared the mean net-approval rating for the week before a certain vote with the mean for the week after the vote. I wanted to look at different types of votes to determine which have the largest, if any, impact. Aslo, since the Republican Trump Score seemed to have the best correlation, I ran the same test with just the Republican Trump Score.

Nominations

This was run with the nominations of Betsy Devos for Secretary of Education, Jeff Sessions for Attorney General, Niel Gorsuch for the Supreme Court, Mike Pompeo for Secretary of State, Gina Haspel for CIA Director, Brett Kavannaugh for the Supreme Court, and William Barr for Attorney General.

For the first two nominations, Trump was coming down from the initial grace period, which explains the downward slide after Sessions and Devos, even though they were not the most popular nominees. There was a slight bump after both Gorsuch and Kavannaugh, although Gorsuch gave a larger increase, probably because he was not a controversial nominee. The William Barr nomination was shortly after the government shutdown, so the President's approval rating was coming out of a downward slide, which explains the increase.

The Republican dataframe does not show much correlation either, although this Trump Score does correlate with the overall Trump Score.

In [23]:
```
nomination = {'Devos':20170207, 'Sessions':20170208, 'Gorsuch':20170407, 'Pom
netAppChange = {}
for i in list(nomination.keys()):
    netAppChange[i] = {'Change': timeApproval(nomination[i], days = 7)['After
pd.DataFrame(netAppChange)
```

Out[23]:

	Devos	Sessions	Gorsuch	Pompeo	Haspel	Kavannaugh	Barr
Change	-3.44	-2.98	1.24	1.17	-0.2	0.19	2.58
Trump_Score	50.93	52.04	54.00	55.00	55.0	52.00	52.00

In [32]:
```
nomination = {'Devos':20170207, 'Sessions':20170208, 'Gorsuch':20170407, 'Pom
netAppChange = {}
for i in list(nomination.keys()):
    netAppChange[i] = {'Change': timeApproval(nomination[i], days = 7)['After
pd.DataFrame(netAppChange)
```

Out[32]:

	Devos	Sessions	Gorsuch	Pompeo	Haspel	Kavannaugh	Barr
Change	-3.44	-2.98	1.24	1.17	-0.20	0.19	2.58
Trump_Score	96.43	100.00	98.08	92.16	98.04	94.12	90.57

Immigration

Again, there is not much correlation between the Trump Score and the change in net-approval rating. However, for the Republican dataframe, as Republicans vote more with the President on immigration, the net-approval rating increases. This is not strong evidence, since there is only a sample size of 3, and there are many other factors that could affect Trump's net-approval.

In [24]:
```
immigration = {'Republican_Bill': 20180627, 'Senate_Immigration':20180215,'Em
netChange = {}
for i in list(immigration.keys()):
    netChange[i] = {'Change': timeApproval(immigration[i], days = 7)['After_M
pd.DataFrame(netChange)
```

Out[24]:

	Republican_Bill	Senate_Immigration	Emergency_Declaration_Block
Change	-1.21	0.06	2.58
Trump_Score	63.32	44.50	52.00

In [33]:
```
immigration = {'Republican_Bill': 20180627, 'Senate_Immigration':20180215,'Em
netChange = {}
for i in list(immigration.keys()):
    netChange[i] = {'Change': timeApproval(immigration[i], days = 7)['After_M
pd.DataFrame(netChange)
```

Out[33]:

	Republican_Bill	Senate_Immigration	Emergency_Declaration_Block
Change	-1.21	0.06	2.58
Trump_Score	68.94	76.47	90.57

Foreign Policy

With only three data points for the Trump Score overall and the Republican Trump Score, there is little correlation. This is probably because the average voter is not extremely interested in whether there are Russian Santions or whether we are involved in the conflict in Yemen. These changes in approval rating are most likely due to other factors outside of Congressional support.

In [25]:
```
# discard senate due to another bill at the same time
foreign = {'House_Russia_Sanctions':20170614, 'House_Yemen':20181218, 'Senate
netForeign = {}
for i in list(foreign.keys()):
    netForeign[i] = {'Change': timeApproval(foreign[i], days = 7)['After_Mean
pd.DataFrame(netForeign)
```

Out[25]:

	House_Russia_Sanctions	House_Yemen	Senate_Yemen
Change	0.67	-1.13	2.68
Trump_Score	23.00	86.00	41.20

In [34]:
```
# discard senate due to another bill at the same time
foreign = {'House_Russia_Sanctions':20170614, 'House_Yemen':20181218, 'Senate
netForeign = {}
for i in list(foreign.keys()):
    netForeign[i] = {'Change': timeApproval(foreign[i], days = 7)['After_Mean
pd.DataFrame(netForeign)
```

Out[34]:

	House_Russia_Sanctions	House_Yemen	Senate_Yemen
Change	0.67	-1.13	2.68
Trump_Score	36.54	78.43	90.36

Major Legislation

Unlike foreign policy, voters do care about their healthcare and taxes. There is correlation on the overall dataframe, but the correlation is most prominent on the Republican votes.

When the Republicans vote with the Trump administration at a percentage less than 90, there is a consistent decrease in the President's net-approval rating, with the net-approval decreasing more as the Republican Trump Score decreases. When the Republican Trump Score is around 95%, there is a modest increase of about 0.6% in the President's net-approval rating.

In [26]:
```
normalLeg = {'ACA_House':20170504,'ACA_Senate':20170728,'Tax_House':20171116,
netLeg = {}
for i in list(normalLeg.keys()):
    netLeg[i] = {'Change': timeApproval(normalLeg[i], days = 7)['After_Mean']
pd.DataFrame(netLeg)
```

Out[26]:

	ACA_House	ACA_Senate	Tax_House	Tax_Senate_Initial	Tax_Senate_Final
Change	-0.10	-2.01	0.58	-1.13	0.60
Trump_Score	70.38	42.00	58.66	45.00	55.74

In [35]:
```
normalLeg = {'ACA_House':20170504,'ACA_Senate':20170728,'Tax_House':20171116,
netLeg = {}
for i in list(normalLeg.keys()):
    netLeg[i] = {'Change': timeApproval(normalLeg[i], days = 7)['After_Mean']
pd.DataFrame(netLeg)
```

Out[35]:

	ACA_House	ACA_Senate	Tax_House	Tax_Senate_Initial	Tax_Senate_Final
Change	-0.10	-2.01	0.58	-1.13	0.60
Trump_Score	86.85	80.77	94.98	84.62	94.14

Correlation

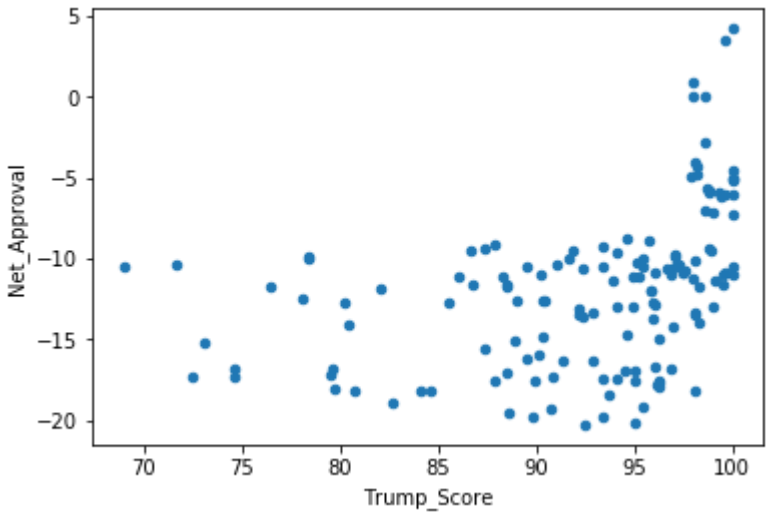
I ran a scatterplot with the Republican Trump Score over the course of the Donald Trump Presidency. There were a few outliers, so this only shows when the Trump Score was above 50 on a given day.

Unfortunately, the high Trump Scores matched with high net-approval ratings are at the beginning of the Trump Presidency, when Republicans vote with the administration often on Cabinet appointees. Otherwise, there is only a slight cluster where around a -10 net-approval and 95-100 Trump Score. However, there are also Republican votes between 70 and 80 where his net-approval is around -10.

This might have more correlation for highly publicized votes. However, this would take more research and extensive amounts of hard-coding to pull out just these votes. Some days would even have to be split when there are multiple votes on that day.

In [76]:
```
aggNew = aggScore(dayWeight = 1, onlyVote = True, party = 'R')
first = pd.DataFrame(graphNetScore(net, aggNew, scatter = True))
nextFrame = first.transpose()
final = nextFrame[ nextFrame['Trump_Score'] >= 50]
final.plot(kind = 'scatter', x = 'Trump_Score', y = 'Net_Approval')
```

Out[76]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x258de0755c0>
```



Conclusion

Although no hard results said that the President's net-approval rating was influenced heavily by Congressional voting, there were some interesting things that I learned and discovered over the course of this project.

1. The Trump Score of both parties decreased when the 116th Congress came into session. This was during the longest government shutdown in history, which probably had an effect.
2. The approval rating of the President is influenced by many factors, and even an influence as large as Congress does not influence President Trump's net-approval rating greatly by itself.
3. Fitting data that is volatile and infrequent can be challenging, and even though it can be made to look nice on a graph, that does not guarantee there is actually strong correlation.

4. This only scratches the surface when trying to discover political trends based on quantitative data. Numerous factors and exhaustive research would be needed to make this model truly accurate.

In [ ]: